

6장 업캐스팅과 다운캐스팅...

I . 업캐스팅과 다운캐스팅

1. 업캐스팅에 대한 설명으로 올바른 것은?

- ① 슈퍼 클래스 객체를 서브 클래스 타입으로 변환한다.
- ② 서브 클래스 객체를 슈퍼 클래스 타입으로 변환한다.
- ③ 서로 상속 관계가 없는 객체를 변환한다.
- ④ 기본 자료형을 객체 타입으로 변환한다.

2. 다음 코드에서 발생하는 타입 변환은?

```
class Person { }  
class Student extends Person { }  
  
Student s = new Student();  
Person p = s;
```

- ① 다운캐스팅
- ② 업캐스팅
- ③ 오버라이딩
- ④ 오버로딩

3. 업캐스팅에 대한 설명으로 올바른 것은?

- ① 반드시 명시적 타입 변환이 필요하다.
- ② 자동 타입 변환이 가능하다.
- ③ 상속 관계가 없어도 가능하다.
- ④ 기본 자료형에서만 가능하다.

4. 업캐스팅된 레퍼런스로 직접 접근할 수 있는 멤버는?

- ① 서브 클래스에 새로 추가된 멤버만
- ② 슈퍼 클래스의 멤버
- ③ 모든 클래스의 멤버
- ④ static 멤버만

5. 다음 코드에서 p의 선언 타입은?

```
Student s = new Student();  
Person p = s;
```

- ① Student
- ② Person
- ③ Object

④ String

6. 다음 중 업캐스팅이 가능한 코드는?

```
class Person { }  
class Student extends Person { }
```

- ① `Student s = new Person();`
- ② `Person p = new Student();`
- ③ `Student s = new Person();`
- ④ `Person p = new String();`

7. 다운캐스팅에 대한 설명으로 올바른 것은?

- ① 서브 클래스 객체를 슈퍼 클래스 타입으로 변환한다.
- ② 슈퍼 클래스 타입의 레퍼런스를 서브 클래스 타입으로 변환한다.
- ③ 정수를 실수로 변환한다.
- ④ 클래스의 상속을 제거한다.

8. 다운캐스팅할 때 필요한 것은?

- ① 자동 타입 변환
- ② 명시적 타입 변환
- ③ new 연산자 제거
- ④ static 키워드

9. 다음 코드의 빈칸에 들어갈 내용으로 올바른 것은?

```
Person p = new Student();  
Student s = _____ p;
```

- ① `(Person)`
- ② `(Student)`
- ③ `(Object)`
- ④ `(String)`

10. 다음 코드에서 `p = new Student();`에 해당하는 것은?

```
Person p = new Student();  
Student s = (Student)p;
```

- ① 업캐스팅
- ② 다운캐스팅
- ③ 오버라이딩
- ④ 동적 바인딩

11. 위 코드에서 `Student s = (Student)p;`에 해당하는 것은?

- ① 업캐스팅
- ② 다운캐스팅
- ③ 오버로딩
- ④ 상속 해제

12. 다음 중 다운캐스팅을 올바르게 표현한 것은?

- ① `Student s = p;`
- ② `Student s = (Student)p;`
- ③ `Person s = (Person)p;`
- ④ `Student s = new Person();`

13. 서브 클래스 객체를 슈퍼 클래스 객체로 취급할 수 있는 이유로 가장 적절한 것은?

- ① 서브 클래스 객체가 슈퍼 클래스의 멤버를 가지고 있기 때문이다.
- ② 모든 클래스의 이름이 같기 때문이다.
- ③ 생성자가 동일하기 때문이다.
- ④ 모든 객체가 같은 필드만 가지기 때문이다.

14. 다음 코드에서 `p`가 실제로 가리키는 객체의 타입은?

```
Person p = new Student();
```

- ① Person
- ② Student
- ③ String
- ④ Object만 가능

15. 업캐스팅된 레퍼런스에 대한 설명으로 올바른 것은?

- ① 실제 객체가 반드시 슈퍼 클래스 객체로 변경된다.
- ② 레퍼런스의 타입은 슈퍼 클래스 타입이다.
- ③ 서브 클래스의 모든 멤버에 직접 접근할 수 있다.
- ④ 명시적 형변환이 반드시 필요하다.

II. instanceof와 객체 타입 판별

16. 레퍼런스가 가리키는 객체의 타입을 식별할 때 사용하는 연산자는?

- ① extends
- ② instanceof
- ③ super
- ④ this

17. `instanceof` 연산의 결과 자료형은?

- ① int
- ② double
- ③ boolean
- ④ String

18. 다음 중 `instanceof` 의 올바른 사용 형식은?

- ① 클래스타입 `instanceof` 객체레퍼런스
- ② 객체레퍼런스 `instanceof` 클래스타입
- ③ `instanceof` (객체레퍼런스)
- ④ 객체레퍼런스.`instanceof` ()

19. 다음 코드의 결과는?

```
class Person { }
class Student extends Person { }

Person p = new Student();
System.out.println(p instanceof Student);
```

- ① true
- ② false
- ③ 0
- ④ 컴파일 오류

20. 다음 코드의 결과는?

```
Person p = new Student();
System.out.println(p instanceof Person);
```

- ① true
- ② false
- ③ null
- ④ Student

21. 다음 상속 관계에서 `Professor` 객체에 대해 true가 되는 것은?

```
class Person { }
class Researcher extends Person { }
class Professor extends Researcher { }
```

- ① `p instanceof Professor` 만 true
- ② `p instanceof Researcher` 만 true
- ③ `p instanceof Person` 만 true
- ④ Person, Researcher, Professor 타입 모두 true

22. 다음 코드의 출력 결과는?

```
Person p = new Student();

if (p instanceof Student)
    System.out.print("Student");
```

- ① Person
- ② Student
- ③ true
- ④ 아무것도 출력되지 않는다.

23. 업캐스팅된 레퍼런스가 가리키는 실제 객체의 타입을 확인하기 어려운 이유는?

- ① 슈퍼 클래스는 여러 서브 클래스에 상속될 수 있기 때문이다.
- ② 객체에는 타입이 없기 때문이다.
- ③ 자바에는 클래스가 없기 때문이다.
- ④ 모든 객체가 같은 타입이기 때문이다.

24. `instanceof` 를 사용하는 주된 목적은?

- ① 객체를 생성하기 위해
- ② 객체의 실제 타입을 식별하기 위해
- ③ 생성자를 호출하기 위해
- ④ 메소드를 삭제하기 위해

25. 다음 상속 관계에서 `new Student()` 객체에 대한 설명으로 올바른 것은?

```
class Person { }
class Student extends Person { }
```

- ① Student 타입이지만 Person 타입은 아니다.
- ② Person 타입이지만 Student 타입은 아니다.
- ③ Student 타입이면서 Person 타입이기도 하다.
- ④ 어떠한 타입에도 해당하지 않는다.

III. 메소드 오버라이딩

26. 메소드 오버라이딩이란?

- ① 같은 클래스에서 같은 이름의 메소드를 여러 개 작성하는 것
- ② 슈퍼 클래스의 메소드를 서브 클래스에서 재정의하는 것
- ③ 메소드의 이름을 변경하는 것
- ④ 생성자를 여러 개 작성하는 것

27. 메소드 오버라이딩 시 슈퍼 클래스 메소드와 동일해야 하는 것으로 가장 적절한 것은?

- ① 메소드 이름만
- ② 리턴 타입만
- ③ 이름, 매개변수 타입과 개수, 리턴 타입
- ④ 메소드 내부의 코드

28. 메소드 오버라이딩을 다른 표현으로 설명한 것은?

- ① 메소드 무시하기 또는 덮어쓰기
- ② 객체 생성하기
- ③ 변수 초기화하기
- ④ 패키지 생성하기

29. 메소드 오버라이딩과 관련하여 발생하는 바인딩은?

- ① 정적 바인딩만
- ② 동적 바인딩
- ③ 데이터 바인딩
- ④ 배열 바인딩

30. 다음 코드에서 오버라이딩된 메소드는?

```
class Shape {
    public void draw() {
        System.out.println("Shape");
    }
}

class Line extends Shape {
    public void draw() {
        System.out.println("Line");
    }
}
```

- ① Shape 클래스의 생성자
- ② Line 클래스의 draw()
- ③ Shape 클래스 자체
- ④ Line 클래스 자체

31. 다음 코드의 출력 결과는?

```
Line line = new Line();
line.draw();
```

- ① Shape
- ② Line
- ③ draw
- ④ 아무것도 출력되지 않는다.

32. 다음 코드의 출력 결과는?

```
Shape shape = new Line();
shape.draw();
```

- ① Shape
- ② Line
- ③ Shape Line
- ④ 컴파일 오류

33. 위 코드에서 `Shape shape = new Line();`에 발생한 타입 변환은?

- ① 다운캐스팅
- ② 업캐스팅
- ③ 오버로딩
- ④ 기본 타입 변환

34. 업캐스팅된 레퍼런스로 오버라이딩된 메소드를 호출하면 실행되는 것은?

- ① 항상 슈퍼 클래스의 메소드
- ② 실제 객체에 오버라이딩된 메소드
- ③ 생성자
- ④ main() 메소드

35. 다음 코드의 출력 결과는?

```
class Shape {
    public void draw() {
        System.out.println("Shape");
    }
}

class Circle extends Shape {
    public void draw() {
        System.out.println("Circle");
    }
}

Shape s = new Circle();
s.draw();
```

- ① Shape
- ② Circle
- ③ Shape Circle
- ④ 컴파일 오류

36. 오버라이딩을 통해 실현되는 객체 지향의 특성은?

- ① 캡슐화
- ② 다형성

- ③ 추상화만
- ④ 패키지화

37. 오버라이딩은 어느 시점의 다형성을 실현하는가?

- ① 컴파일 시간
- ② 실행 시간
- ③ 클래스 작성 시간
- ④ 객체 삭제 시간

38. 오버라이딩된 메소드가 실행 중 실제 객체에 따라 선택되는 것을 무엇이라고 하는가?

- ① 정적 바인딩
- ② 동적 바인딩
- ③ 업캐스팅
- ④ 다운캐스팅

39. 다음 코드의 출력 결과는?

```
class Shape {
    public void draw() {
        System.out.print("Shape");
    }
}

class Rect extends Shape {
    public void draw() {
        System.out.print("Rect");
    }
}

Shape p = new Rect();
p.draw();
```

- ① Shape
- ② Rect
- ③ ShapeRect
- ④ 출력 없음

40. 다음 `paint()` 메소드에서 `p.draw()` 가 호출할 메소드를 결정하는 기준은?

```
static void paint(Shape p) {
    p.draw();
}
```

- ① 변수 이름
- ② p가 실제로 가리키는 객체
- ③ 클래스 파일 이름
- ④ main()의 위치

IV. 동적 바인딩과 다형성

41. 동적 바인딩에 대한 설명으로 올바른 것은?

- ① 실행할 메소드를 컴파일 전에 항상 결정한다.
- ② 실행할 메소드를 실행 시간에 결정한다.
- ③ 객체의 필드를 삭제한다.
- ④ 클래스의 상속 관계를 제거한다.

42. 다음 코드의 출력 결과는?

```
Shape p = new Line();  
p.draw();
```

Line 클래스가 draw() 를 오버라이딩하여 "Line" 을 출력한다고 할 때 정답은?

- ① Shape
- ② Line
- ③ ShapeLine
- ④ 오류

43. 다음 코드에서 paint(new Rect()); 의 출력 결과는?

```
static void paint(Shape p) {  
    p.draw();  
}
```

Rect 클래스의 draw() 가 "Rect" 를 출력한다.

- ① Shape
- ② Line
- ③ Rect
- ④ Circle

44. 다음 코드의 출력 순서로 올바른 것은?

```
paint(new Shape());  
paint(new Line());  
paint(new Rect());  
paint(new Circle());
```

- ① Shape → Shape → Shape → Shape
- ② Shape → Line → Rect → Circle
- ③ Line → Rect → Circle → Shape
- ④ Circle → Rect → Line → Shape

45. 다음 중 실행 시간 다형성을 실현하는 것은?

- ① 메소드 오버라이딩
- ② 메소드 오버로딩
- ③ 변수 선언
- ④ 배열 선언

46. 메소드 오버로딩이 실현하는 다형성의 시점은?

- ① 실행 시간
- ② 컴파일 시간
- ③ 객체 소멸 시간
- ④ 가비지 컬렉션 시간

47. 오버라이딩의 목적으로 가장 적절한 것은?

- ① 서브 클래스마다 같은 이름의 메소드를 자신에게 맞게 구현한다.
- ② 모든 서브 클래스가 동일한 결과만 출력하도록 한다.
- ③ 상속을 사용하지 못하도록 한다.
- ④ 객체 생성을 막는다.

48. Shape의 `draw()` 를 Line, Rect, Circle에서 각각 재정의한 경우 같은 `draw()` 호출이 서로 다른 결과를 만드는 특성은?

- ① 캡슐화
- ② 상속 제거
- ③ 다형성
- ④ 가비지 컬렉션

V. super

49. `super` 가 가리키는 것은?

- ① 현재 클래스의 객체
- ② 슈퍼 클래스의 멤버
- ③ 현재 메소드의 지역 변수
- ④ `main()` 메소드

50. `super` 를 사용할 수 있는 곳은?

- ① 서브 클래스
- ② 상속 관계가 없는 모든 클래스
- ③ 인터페이스에서만
- ④ `main()`에서만

51. 서브 클래스에서 슈퍼 클래스의 메소드를 호출할 때 사용할 수 있는 키워드는?

- ① `this`
- ② `new`

- ③ super
- ④ instanceof

52. 다음 코드에서 `super.draw();` 가 호출하는 메소드는?

```
class Shape {
    public void draw() {
        System.out.println("Shape");
    }
}

class Circle extends Shape {
    public void draw() {
        super.draw();
    }
}
```

- ① Circle의 draw()
- ② Shape의 draw()
- ③ main()
- ④ 생성자

53. `super` 를 이용한 슈퍼 클래스 멤버 접근은 어떤 바인딩으로 처리되는가?

- ① 동적 바인딩
- ② 정적 바인딩
- ③ 자동 바인딩이 없다.
- ④ 배열 바인딩

54. 다음 코드의 출력 결과는?

```
class Shape {
    protected String name = "Shape";
}

class Circle extends Shape {
    protected String name = "Circle";

    void print() {
        System.out.println(super.name);
    }
}
```

- ① Circle
- ② Shape
- ③ null
- ④ 컴파일 오류

VI. 오버라이딩과 오버로딩

55. 오버로딩에 대한 설명으로 올바른 것은?

- ① 슈퍼 클래스의 메소드를 서브 클래스에서 재정의한다.
- ② 같은 이름의 여러 메소드를 중복 작성한다.
- ③ 객체를 슈퍼 클래스 타입으로 변환한다.
- ④ 객체의 실제 타입을 검사한다.

56. 오버라이딩에 대한 설명으로 올바른 것은?

- ① 같은 클래스에서 매개변수가 다른 메소드를 여러 개 작성한다.
- ② 상속 관계에서 슈퍼 클래스의 메소드를 서브 클래스에서 재정의한다.
- ③ 메소드 이름이 반드시 달라야 한다.
- ④ 상속과 관계가 없다.

57. 오버로딩의 조건으로 올바른 것은?

- ① 메소드 이름은 같고 매개변수의 개수나 타입이 달라야 한다.
- ② 메소드 이름이 반드시 달라야 한다.
- ③ 매개변수가 반드시 같아야 한다.
- ④ 슈퍼 클래스와 서브 클래스에서만 가능하다.

58. 오버라이딩의 조건으로 올바른 것은?

- ① 메소드 이름만 같으면 된다.
- ② 메소드 이름, 매개변수 타입과 개수, 리턴 타입 등이 동일해야 한다.
- ③ 매개변수 개수가 반드시 달라야 한다.
- ④ 리턴 타입만 다르면 된다.

59. 오버로딩과 오버라이딩의 바인딩을 올바르게 연결한 것은?

- ① 오버로딩-동적 바인딩 / 오버라이딩-정적 바인딩
- ② 오버로딩-정적 바인딩 / 오버라이딩-동적 바인딩
- ③ 둘 다 정적 바인딩
- ④ 둘 다 동적 바인딩

60. 다음 코드의 실행 결과는?

```
class Weapon {
    protected int fire() {
        return 1;
    }
}

class Cannon extends Weapon {
    protected int fire() {
        return 10;
    }
}

Weapon weapon = new Cannon();
System.out.println(weapon.fire());
```

- ① 0
- ② 1
- ③ 10
- ④ 컴파일 오류

4지선다형 60문항 정답

문항	정답	문항	정답	문항	정답
1	②	21	④	41	②
2	②	22	②	42	②
3	②	23	①	43	③
4	②	24	②	44	②
5	②	25	③	45	①
6	②	26	②	46	②
7	②	27	③	47	①
8	②	28	①	48	③
9	②	29	②	49	②
10	①	30	②	50	①
11	②	31	②	51	③
12	②	32	②	52	②
13	①	33	②	53	②
14	②	34	②	54	②
15	②	35	②	55	②
16	②	36	②	56	②
17	③	37	②	57	①
18	②	38	②	58	②
19	①	39	②	59	②
20	①	40	②	60	③

4지선다형 60문항 해설

1~15. 업캐스팅과 다운캐스팅

1. ② 서브 클래스 객체를 슈퍼 클래스 타입으로 변환한다.

업캐스팅은 서브 클래스의 객체를 슈퍼 클래스 타입으로 변환하는 것이다.

2. ② 업캐스팅

`Student` 는 `Person` 을 상속하므로 `Student` 객체를 `Person` 타입의 레퍼런스에 대입하는 것은 업캐스팅이다.

3. ② 자동 타입 변환이 가능하다.

업캐스팅은 별도의 명시적 타입 변환 없이 자동으로 이루어진다. 자료에서도 `Person p = s;` 를 자동 타입 변환으로 설명한다.

4. ② 슈퍼 클래스의 멤버

업캐스팅된 레퍼런스로는 객체 내부의 슈퍼 클래스 멤버에 접근할 수 있다.

5. ② Person

`Person p` 에서 레퍼런스 변수 `p` 의 선언 타입은 `Person` 이다. 실제로 가리키는 객체는 `Student` 객체이다.

6. ② `Person p = new Student();`

`Student` 가 `Person` 의 서브 클래스이므로 `Student` 객체를 `Person` 타입의 레퍼런스로 가리킬 수 있다.

7. ② 슈퍼 클래스 타입의 레퍼런스를 서브 클래스 타입으로 변환한다.

다운캐스팅은 업캐스팅된 슈퍼 클래스 타입의 레퍼런스를 다시 서브 클래스 타입으로 변환하는 것이다.

8. ② 명시적 타입 변환

다운캐스팅에서는 `(Student)` 와 같이 개발자가 명시적으로 타입을 표시해야 한다.

9. ② `(Student)`

`Person` 타입의 레퍼런스 `p` 를 `Student` 타입으로 다운캐스팅하므로 `(Student)p` 라고 작성한다.

10. ① 업캐스팅

`new Student()` 객체를 `Person` 타입의 `p` 가 가리키므로 업캐스팅이다.

11. ② 다운캐스팅

`Student s = (Student)p;` 는 `Person` 타입 레퍼런스를 `Student` 타입으로 명시적으로 변환한다.

12. ② `Student s = (Student)p;`

다운캐스팅은 서브 클래스 타입을 괄호 안에 명시하여 변환한다.

13. ① 서브 클래스 객체가 슈퍼 클래스의 멤버를 가지고 있기 때문이다.

서브 클래스 객체는 슈퍼 클래스의 멤버를 모두 가지므로 슈퍼 클래스 객체로 취급할 수 있다.

14. ② Student

`p` 의 선언 타입은 `Person`이지만 `new Student()` 로 생성된 실제 객체의 타입은 `Student`이다.

15. ② 레퍼런스의 타입은 슈퍼 클래스 타입이다.

업캐스팅 후 `p` 는 `Person` 타입 레퍼런스이지만 실제로는 `Student` 객체를 가리킬 수 있다.

16~25. instanceof와 객체 타입 판별

16. ② instanceof

`instanceof` 는 레퍼런스가 가리키는 객체의 타입을 식별하기 위해 사용하는 연산자이다.

17. ③ boolean

`instanceof` 의 결과는 `true` 또는 `false` 이므로 boolean 값이다.

18. ② 객체레퍼런스 instanceof 클래스타입

자료에 제시된 기본 형식이다.

19. ① true

`p`가 실제로 `Student` 객체를 가리키므로 `p instanceof Student`는 `true`이다.

20. ① true

`Student`는 `Person`을 상속하므로 `Student` 객체는 `Person` 타입이기도 하다.

21. ④ Person, Researcher, Professor 타입 모두 true

`Professor`는 `Researcher`를 상속하고 `Researcher`는 `Person`을 상속한다. 따라서 `Professor` 객체는 `Professor`, `Researcher`, `Person` 타입에 모두 해당한다. 자료의 실행 예에서도 같은 관계를 보여준다.

22. ② Student

조건식 `p instanceof Student`가 참이므로 `System.out.print("Student");`가 실행된다.

23. ① 슈퍼 클래스는 여러 서브 클래스에 상속될 수 있기 때문이다.

`Person` 타입의 레퍼런스는 `Person`뿐 아니라 `Student`, `Researcher`, `Professor` 등의 객체를 가리킬 수 있으므로 레퍼런스 타입만으로 실제 객체 타입을 판단하기 어렵다.

24. ② 객체의 실제 타입을 식별하기 위해

`instanceof`는 업캐스팅된 레퍼런스 등이 실제로 어떤 객체를 가리키는지 판단할 때 사용한다.

25. ③ Student 타입이면서 Person 타입이기도 하다.

`Student`가 `Person`을 상속하므로 `Student` 객체는 `Student`인 동시에 `Person` 타입에도 해당한다.

26~40. 메소드 오버라이딩

26. ② 슈퍼 클래스의 메소드를 서브 클래스에서 재정의하는 것

메소드 오버라이딩은 상속받은 슈퍼 클래스의 메소드를 서브 클래스에서 다시 구현하는 것이다.

27. ③ 이름, 매개변수 타입과 개수, 리턴 타입

오버라이딩에서는 슈퍼 클래스 메소드와 이름, 매개변수 타입과 개수, 리턴 타입 등을 동일하게 작성한다.

28. ① 메소드 무시하기 또는 덮어쓰기

자료에서는 오버라이딩을 '메소드 무시하기', '덮어쓰기'로 번역하기도 한다고 설명한다.

29. ② 동적 바인딩

메소드 오버라이딩이 이루어지면 실제 객체에 따라 오버라이딩된 메소드가 실행되는 동적 바인딩이 발생한다.

30. ② Line 클래스의 draw()

`Line`이 `Shape`의 `draw()`와 동일한 형식으로 `draw()`를 다시 정의했으므로 `Line`의 `draw()`가 오버라이딩된 메소드이다.

31. ② Line

`line`은 `Line` 객체이므로 `Line` 클래스에서 재정의한 `draw()`가 실행되어 `Line`을 출력한다.

32. ② Line

레퍼런스 타입은 `Shape`이지만 실제 객체가 `Line`이므로 동적 바인딩에 의해 `Line`의 `draw()`가 실행된다. 자료의 업캐스팅 호출 사례에서도 결과는 `Line`이다.

33. ② 업캐스팅

`Line` 객체를 `Shape` 타입 레퍼런스에 저장했으므로 업캐스팅이다.

34. ② 실제 객체에 오버라이딩된 메소드

업캐스팅되어 있어도 오버라이딩된 메소드를 호출하면 실제 객체의 메소드가 실행된다. 이것이 동적 바인딩이다.

35. ② Circle

`s`의 선언 타입은 `Shape`이지만 실제 객체는 `Circle`이다. 따라서 `Circle`에서 오버라이딩한 `draw()`가 실행된다.

36. ② 다형성

각 서브 클래스가 동일한 이름의 메소드를 서로 다르게 구현함으로써 다형성을 실현한다.

37. ② 실행 시간

오버라이딩은 동적 바인딩을 통해 **실행 시간 다형성**을 실현한다.

38. ② 동적 바인딩

실행할 메소드를 실제 객체에 따라 실행 시간에 결정하는 것을 동적 바인딩이라고 한다.

39. ② Rect

`p`는 `Shape` 타입이지만 실제 객체는 `Rect`이므로 `Rect`의 `draw()`가 호출되어 `Rect`를 출력한다.

40. ② p가 실제로 가리키는 객체

`p.draw()`에서 어느 `draw()`가 실행되는지는 `p`의 선언 타입이 아니라 실행 시점에 `p`가 실제로 가리키는 객체에 따라 결정된다.

41~48. 동적 바인딩과 다형성

41. ② 실행할 메소드를 실행 시간에 결정한다.

동적 바인딩은 실행할 메소드를 실행 시간(run time)에 결정한다.

42. ② Line

실제 객체가 `Line`이므로 `Line`에서 오버라이딩한 `draw()`가 실행된다.

43. ③ Rect

`new Rect()`가 전달되므로 `p`는 `Rect` 객체를 가리킨다. 따라서 `Rect`의 `draw()`가 실행된다.

44. ② Shape → Line → Rect → Circle

각 객체의 실제 타입에 해당하는 `draw()`가 동적 바인딩에 의해 차례대로 실행된다. 자료의 예제에서도 같은 결과가 제시된다.

45. ① 메소드 오버라이딩

오버라이딩은 실행 시간에 실제 객체에 따라 메소드가 결정되므로 실행 시간 다형성을 구현한다.

46. ② 컴파일 시간

자료에서는 오버라이딩은 실행 시간 다형성, 오버로딩은 컴파일 타임 다형성을 실현한다고 구분한다.

47. ① 서브 클래스마다 같은 이름의 메소드를 자신에게 맞게 구현한다.

슈퍼 클래스가 제공하는 공통 메소드 형식을 유지하면서 각 서브 클래스가 자신에게 필요한 동작으로 재정의하는 것이 목적이다.

48. ③ 다형성

같은 `draw()` 라는 인터페이스를 사용하지만 Line, Rect, Circle에서 서로 다른 동작을 수행하는 것이 다형성이다.

49~54. super

49. ② 슈퍼 클래스의 멤버

`super` 는 서브 클래스에서 슈퍼 클래스의 멤버에 접근할 때 사용하는 레퍼런스이다.

50. ① 서브 클래스

`super` 는 상속 관계의 서브 클래스에서 사용한다.

51. ③ `super`

서브 클래스에서 `super.메소드()` 형태로 슈퍼 클래스의 메소드를 호출할 수 있다.

52. ② Shape의 `draw()`

Circle의 `draw()` 안에서 `super.draw()` 를 실행하면 슈퍼 클래스인 Shape의 `draw()` 가 호출된다.

53. ② 정적 바인딩

자료에서는 컴파일러가 `super` 를 통한 접근을 정적 바인딩으로 처리한다고 설명한다.

54. ② Shape

`super.name` 은 현재 Circle 객체의 `name` 이 아니라 슈퍼 클래스 Shape에 선언된 `name` 에 접근하므로 `Shape` 이 출력된다.

55~60. 오버라이딩과 오버로딩

55. ② 같은 이름의 여러 메소드를 중복 작성한다.

오버로딩은 같은 이름의 메소드를 매개변수의 개수나 타입을 달리하여 여러 개 작성하는 것이다.

56. ② 상속 관계에서 슈퍼 클래스의 메소드를 서브 클래스에서 재정의한다.

오버라이딩은 반드시 상속 관계를 전제로 하며 슈퍼 클래스의 메소드를 서브 클래스에서 다시 구현한다.

57. ① 메소드 이름은 같고 매개변수의 개수나 타입이 달라야 한다.

오버로딩에서는 같은 이름을 사용하면서 매개변수의 개수 또는 타입을 다르게 한다.

58. ② 메소드 이름, 매개변수 타입과 개수, 리턴 타입 등이 동일해야 한다.

오버라이딩은 슈퍼 클래스의 메소드와 동일한 형식을 유지하면서 구현 내용을 재정의한다.

59. ② 오버로딩-정적 바인딩 / 오버라이딩-동적 바인딩

자료의 비교표에서 오버로딩은 호출할 메소드를 컴파일 시 결정하는 정적 바인딩, 오버라이딩은 실행 시간에 오버라이딩된 메소드를 찾아 호출하는 동적 바인딩으로 구분한다.

60. ③ 10

`Weapon weapon = new Cannon();` 은 업캐스팅이다. `Cannon` 이 `fire()` 를 오버라이딩했으므로 `weapon.fire()` 를 호출하면 동적 바인딩에 의해 Cannon의 `fire()` 가 실행되어 `10` 을 반환한다. 자료의 Weapon/Cannon 예제에서도 기본 무기는 1, 대포는 10을 출력한다.